CHAPTER 10

Design A Web Crawler

Politely and efficiently download billions of web pages for search indexing.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a web crawler (spider/robot) that discovers new or updated content on the web - pages, images, videos, PDFs - starting from a set of seed URLs and following hyperlinks. Primary use: **search engine indexing**; others: web archiving, data mining, copyright monitoring.

Why interviewers ask it: Tests BFS at internet scale, queues, politeness, duplicate detection, robustness and extensibility.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Main purpose?	Search engine indexing.
Pages collected per month?	1 billion.
Content types?	HTML only (extensible).
Consider newly added/edited pages?	Yes.
Store HTML? How long?	Yes, up to 5 years.
Duplicate content?	Ignore it.

3. Requirements

Functional

- Crawl pages from seeds by following links.
- Extract links and queue new ones.
- Store content for the indexer.
- Detect duplicates and avoid revisiting.

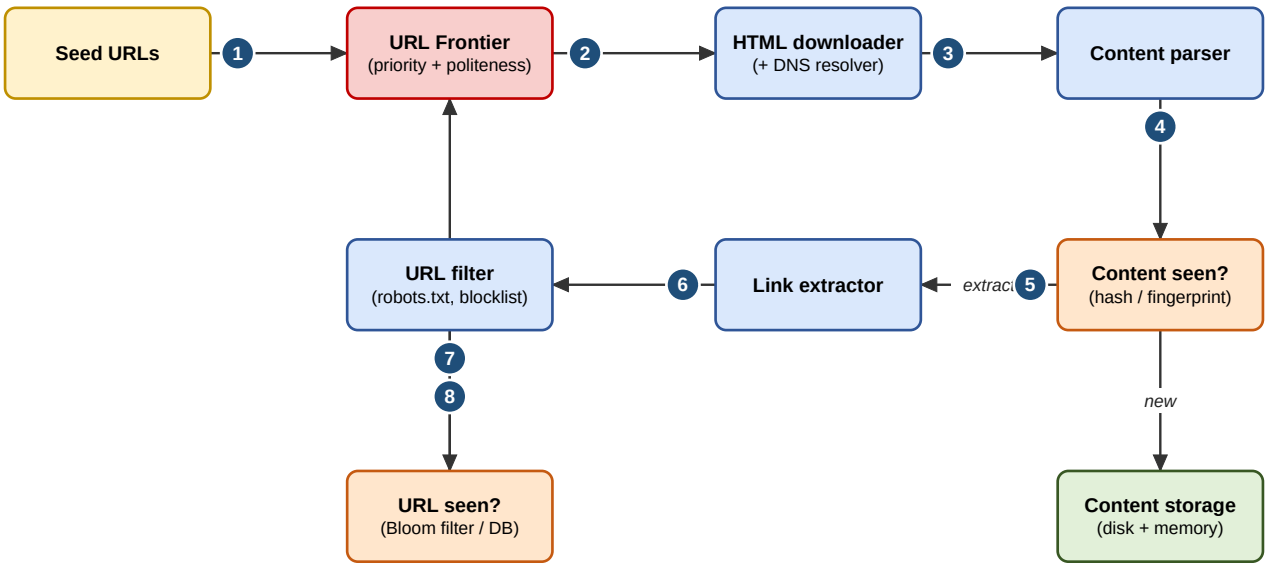
Non-functional

- **Scalability:** parallel / distributed.
- **Robustness:** bad HTML, traps, unresponsive servers, malicious pages.
- **Politeness:** do not overload any host.
- **Extensibility:** add new content types (images, PDF) with minimal change.

4. Back-of-the-envelope estimation

- 1 B pages/month -> about 400 pages/s (1 B / 30 / 86,400 = 385). Peak about 2x = **800 QPS**.
- Average page 500 KB -> 1 B x 500 KB = **500 TB/month**; 5 years = 30 PB (use replication/compression, object storage).
- Bandwidth about 400 x 500 KB = 200 MB/s ingress = about 1.6 Gbps sustained.

5. High-level design



A BFS loop: frontier -> download -> parse -> dedupe -> extract -> filter -> dedupe URLs -> frontier.

Step	What happens
1	Start with **seed URLs** (popular domains, topic-based seeds) and put them in the URL Frontier.
2	The frontier hands out a URL, honouring **politeness** (one request at a time per host, with a delay) and **priority** (important or frequently changing pages first).
3	The HTML downloader resolves DNS (cached), checks **robots.txt**, downloads the page, then the parser validates and cleans the HTML.
4	A **content-seen** check (hash or fingerprint of the page) drops duplicates; about 30% of pages are duplicates.
5	For new content, the **link extractor** pulls every hyperlink and resolves relative URLs to absolute ones.
6	The **URL filter** removes blocked file types, error links, blocklisted sites and spider-trap patterns.
7	A **URL-seen** check (Bloom filter plus store) removes URLs already visited or already in the frontier.
8	Only new URLs go back into the frontier, closing the BFS loop. In parallel, `seen -> store` saves page content for the indexer.

6. Deep dive

URL Frontier: the heart of the crawler

- A FIFO-style queue implementing **BFS**, but with extra structure: priority, freshness, politeness.
- **Politeness:** one request at a time per host. Map each host to a dedicated FIFO queue (mapping table) with a worker thread per queue, plus a delay between downloads of the same host.
- **Priority:** a prioritizer scores URLs (PageRank, traffic, update frequency) into priority queues; a selector picks higher-priority queues more often.
- **Freshness:** recrawl based on update history; prioritise important pages. Frontier is huge -> keep it on disk with in-memory buffers.

HTML downloader

- Respects **robots.txt** (the Robots Exclusion Protocol): fetch and cache it per host, periodically refresh.
- **Performance:** distributed crawl (many servers, each owning a URL partition via consistent hashing), cache DNS results (DNS lookups take 10-200 ms and block), locality (crawl servers geographically close to hosts), short timeouts for unresponsive hosts.

Robustness

- Consistent hashing to add/remove downloaders; save crawl state periodically for recovery; exception handling; data validation.

Duplicate detection

- **Content dedup:** compare hash/fingerprint (e.g. SimHash/checksum) of page content: about 30% of web pages are duplicates.
- **URL dedup:** Bloom filter or a hash set of normalised URLs to avoid re-adding visited URLs.

Spider traps and bad content

- Infinite depth links (e.g. calendars, `/a/b/a/b/...`): set a **max URL length / depth**, per-site page limits, manual blocklist.
- Filter ads, spam, noise via heuristics; avoid data-noise pages.

Server-side rendering and extensibility

- Many sites build links with JavaScript; render pages server-side (headless browser) before parsing.
- Plug-in modules: PNG downloader, web monitor, etc. attach after the downloader without changing the core pipeline.

Storage

- Most content stored on disk/object store (too large for memory); popular content cached in memory. The parsed text is passed to the indexer.

Analytics and ops

- Track crawl rate per host, error rates, queue depth, duplicate rate, bytes downloaded; alert on stuck queues.

7. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
BFS vs DFS	BFS: breadth, standard / DFS: can go very deep (traps)	BFS with priority; DFS rarely appropriate.
Duplicate check	Bloom filter: tiny, false positives / exact hash set: memory heavy	Bloom filter first, authoritative store for confirmation.
Single vs distributed crawl	Single: simple / Many: throughput, locality	Distributed partitioned by host hash.
Freshness vs cost	Recrawl often / save bandwidth	Adaptive recrawl by observed change frequency.

8. Scaling, reliability, failure modes

- Partition by host so politeness is naturally local to one crawler.
- Queue + workers decouple stages; each stage scales independently.
- Retry with backoff; dead-letter URLs failing repeatedly.

9. Likely follow-up questions

- How do you crawl a 10x larger web or recrawl the whole web weekly?
- How do you detect near-duplicate pages?
- How do you avoid hammering a single site?
- How would you handle JavaScript-heavy sites?

10. Common mistakes

- Forgetting politeness and robots.txt.
- Plain FIFO queue with no per-host limit.
- No protection against spider traps.

11. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"I'll assume the purpose is search engine indexing, about 1 billion pages a month, HTML only but extensible, with new and edited pages recrawled and content stored for up to 5 years. That is about 400 pages per second, or 800 at peak, and roughly 500 TB per month at 500 KB per page."
2. High-level design	10 min	"A BFS loop: seed URLs go to the URL Frontier, the downloader fetches pages, the parser validates them, a content-seen check removes duplicates, a link extractor finds new URLs, a filter removes bad ones, a URL-seen check removes visited ones, and new URLs go back to the frontier. Content is stored for the indexer."
3. Deep dive	25 min	"The frontier matters most. Politeness: map each host to its own FIFO queue and a worker with a delay. Priority: score pages by PageRank or update frequency and sample high-priority queues more often. Freshness: recrawl by change history. For the downloader: robots.txt cache, DNS cache, distributed crawling partitioned by host with consistent hashing. For robustness: spider-trap limits, duplicate detection with Bloom filters, saved crawl state for recovery."
4. Wrap up	4 min	"I'd also render JavaScript pages server-side, add plug-in modules for images or PDFs, and monitor crawl rate per host, queue depth and duplicate rate. To crawl 10x more, I'd add partitions and crawler servers."

12. 60-second recap / practice checklist

- Pipeline: seed -> frontier -> downloader -> parser -> content-dedupe -> extractor -> filter -> URL-dedupe -> frontier.
- Frontier handles priority + politeness + freshness; Bloom filters dedupe; DNS cache; consistent hashing across crawlers.
- About 400 pages/s, 500 TB/month.

13. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Mercator: a scalable, extensible web crawler
• Google: robots.txt introduction
• GeeksforGeeks: Design a web crawler